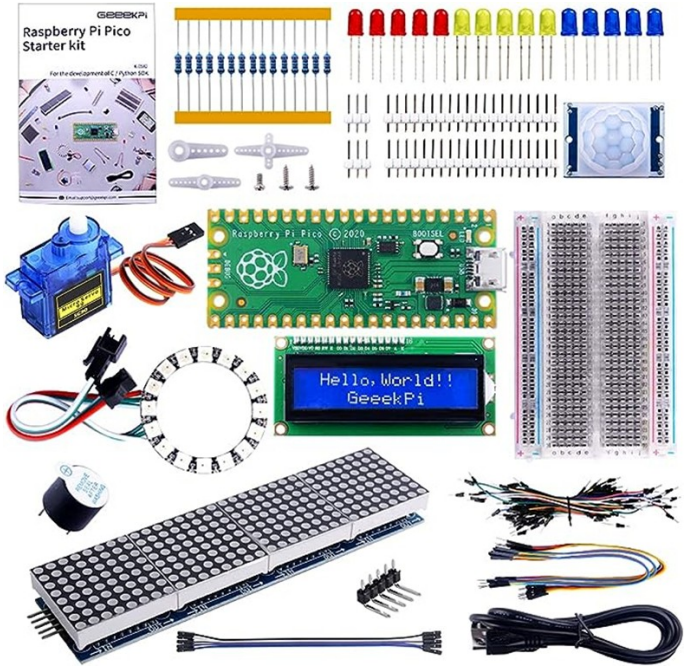
Atelier Raspi N°21

La programmation
Le code Morse

Logo du Raspberry Pico

Logo du MicroPython



L'atelier a pour valeurs, le partage, l'aide, la formation, le faire et construire ensemble à partir de l'expérience des participants

- **La programmation**

  Etape 1 Programmation – Langage MicroPython

  Etape 2 Les variables dans Python

  Etape 3 Les types de variables et opérateurs standards

  Etape 4 Les structures de contrôle 1

  Etape 5 Les structures de contrôle 2

  Etape 6 Les structures de contrôle 3

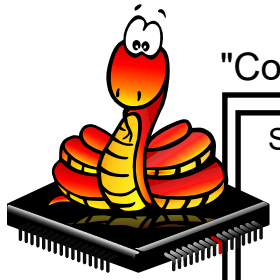
  Etape 7 Listes et dictionnaires

  Etape 8 Fonctions

  Etape 9 Fonctions natives, fonction intégrée à python

  Etape 10 Modules

A21E1 Programmation – Langage MicroPython



"Core" Micropython

- Syntaxe
- Opérateurs
- Mots-clés
- Fonctions "Built-in

Modules standards

cmath	uheapq	ustruct
gc	uio	utime
math	ujson	usys
uarray	uos	uzlib
ubinascii	ure	_thread
ucollections	uselect	
uerrno	usocket	
uhashlib	ussl	

Un module CPython porté en Micropython porte le même nom précédé de **u**
Les modules ainsi portés disposent d'un nombre limité de fonctions mais l'essentiel y est.
Modules essentiels

Modules propres à Micropython

btree	network
framebuf	ubluetooth
machine	ucryptolib
micropython	uctypes

Modules spécifiques cartes ("Ports" Micropython)

- Pyboard: pyb
- Pi Pico: machine, rp2
- ESP: esp
- Wipy: wipy

Etc.
Plusieurs 10aine ports disponibles

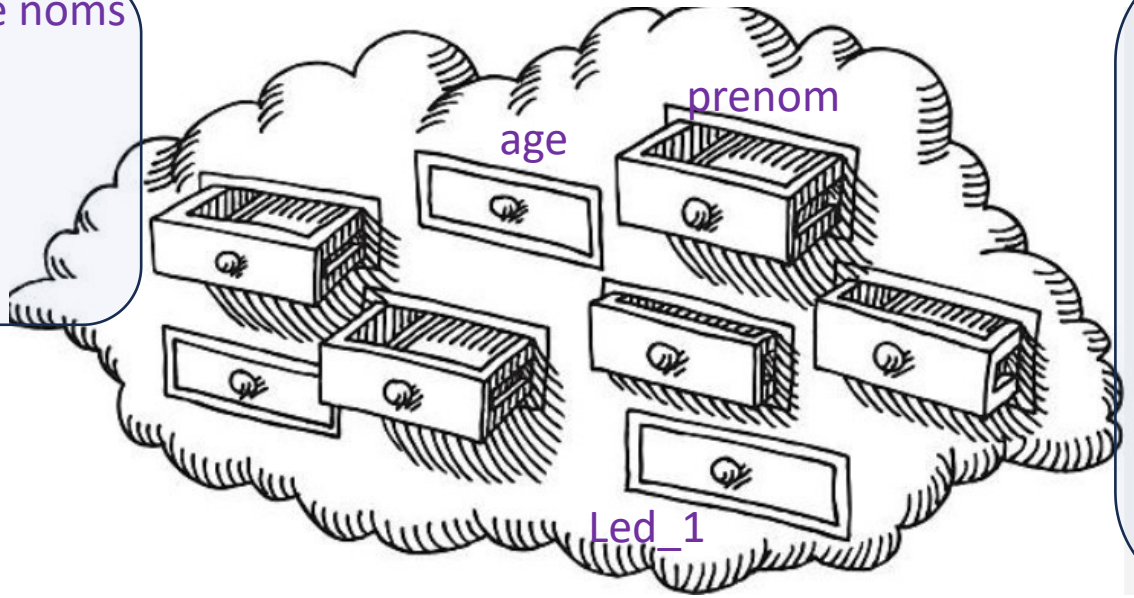
Modules "librairies" hardware

1-wire	Neopixel
DHT	...
LCD	...
OLED	...

A21E2 Les variables

Une variable est un espace mémoire dans lequel on va stocker des objets.
C'est comme un tiroir ou l'on peut ranger (écrire), regarder (lire) ou enlever (supprimer) un ou plusieurs objets.
On va donner un nom à cette variable afin de pour pouvoir l'identifier, comme on met une étiquette sur un tiroir.
Ce nom doit suivre quelques règles :
- être compréhensible : décrire à quoi sert ce tiroir
- commencer par un caractère et non par un chiffre
- ne pas contenir de caractère spécial ni accentué
- ne pas contenir d'espace, si l'on souhaite donner un nom composé, utiliser le caractère souligné (underscore) '_'
- il faut rester cohérent dans la façon de nommer les variables

Exemples de noms de variable:
age
led_1
led_2



Affectation:
Pour écrire un objet dans une variable, on utilise l'opérateur '=' en mettant le nom de la variable à gauche de l'opérateur et l'objet à associé à droite.
Exemple:
age = 10
On peut lire le contenu de notre variable pour l'afficher:
print(age)

A21E3 Les types de variables et opérateurs standards

Une variable sans valeur (tiroir vide) est définie par : *age = None*
Type (objet) de base:
- ***bool*** : définit un booléen qui peut prendre les valeurs ***True*** ou ***False*** *(Vrai ou faux)*
- ***int*** : définit un nombre entier (ex: 10)
- ***float*** : définit un nombre réel (ex :10,2)
- ***str*** : définit une chaîne de caractère (string, ex : « Bonjour tout le monde ! »)

Lors d'un test les valeurs suivantes sont considérée comme fausse : ***False, None, '', [], (), {}***

Opération mathématique:
- ***+*** : addition, ex: ***10+3*** -> 13
- ***-*** : sustraction, ex: ***10-3*** -> 7
- ******* : multiplication, ex: ***5*2*** -> 10
- ***/*** : division, ex: ***20/2*** -> 10
- ******** : élévation a la puissance, ex: ***6**4*** -> 1296
- ***//*** : division entière, ex: ***6//4*** -> 1
- ***%*** : reste de la division entière, ex: ***6%4*** -> 2

Opérateurs de comparaison:
- ***==*** : égalité, ***None==None*** est vrai
- ***!=*** : inégalité, ***None!=None*** est faux
- ***>, >=, <, <=*** : comparaison, ***6>3*** est vrai
- ***is, is not*** : permet de comparer l'identité des objets

Opérateurs booléens:
- ***and : True and False est faux***
- ***or : True and False est vrai***
- ***not*** : ***not False*** est vrai

A21E4 Les structures de contrôle- 1

Indentation

En python l'indentation est obligatoire et permet de définir un bloc d'une ou plusieurs instructions,
Une indentation est un retrait du code de 4 caractères, elle peut être insérée par 4 'espace' ou par la touche tabulation 'TAB'
Le symbole ':' à la fin d'une ligne d'instruction marque le début d'un bloc.

Seules quelques instructions permettent de définir le début d'un bloc:
- ***def*** : définition d'une fonction
- **if :** début d'un test
- **while et for** : début d'une boucle

A21E5 Les structures de contrôle- 2

Comparaison

Sert à comparer deux valeurs, le résultat d'une comparaison est un booléen qui prend la valeur ***True*** ou ***False***.
< : strictement inférieur
<= : inférieur ou égal
== : égal
!= : différent
> : strictement supérieur
>= : supérieur ou égal
Exemple : **3 > 2** -> ***True***

L'instruction if, if…else, if…elif…else

L'instruction ***if*** Permet d'exécuter un bloc si la condition est vraie (***True***)**.** Si ce n'est pas le cas on peut continuer le programme sans traiter le bloc d'instruction ou décider de traiter un autre bloc d'instruction délimité soit par ***elif*** si on souhaite rendre ce nouveau bloc conditionnel, soit par ***else*** si on souhaite le traiter par défaut.
Exemple:

```
n = -3
print(f 'le nombre {n} est: ')
if n > 0:
    print('positif')
elif n < 0:
    print('négatif')
else :
    print('zéro')
```

A21E6 Les structures de contrôle- 3

Boucle while

La boucle ***while*** exécute un bloc tant que la condition est vraie (***True***).
Exemple:

```
n = 3
while n > 0:
    print(n)
    n = n – 1
print('fini !')
```

Boucle for

La boucle **for** permet de parcourir une liste d'objet une à une dans l'ordre de la première à la dernière.
Exemple:

```
for c in 'hello':
    print(c)
```

la variable c va prendre tour à tour les valeurs 'h', 'e', 'l', 'l', 'o'

A21E7 Listes et dictionnaires

Listes

La liste est une séquence linéaire d'objet. Elle est délimitée par des ***[] ou des ()*** et ses éléments sont séparés par des ***','***. Une liste peut contenir plusieurs types d'objet. On peut utiliser le constructeur ***list()*** pour créer une liste vide,
Exemple:
N = [123, 25, 12]
B = ['chat', 12, 'chien']
La fonction ***len*** permet de connaître la longueur d'une liste (son nombre d'élément):
len(N) retourne la valeur 3
On peut parcourir une liste par itération:

```
for item in N:
    print(item)
```

Ou par index:

```
for index in range(len(N)):
    print(N[index])
```

Dictionnaires

Comme les listes les dictionnaires peuvent contenir plusieurs objets de type différents. Ceux-ci seront toujours sous la forme d'une paire ***key:valeur***. Le constructeur ***dict()*** permet de créer un dictionnaire vide. Un dictionnaire est délimité par ***{}***.
Exemple:
N = {'voiture':'4 roues', 'moto':'2 roues'}
B = {'animal':'chat', 'taille_poids':[60,10]}
On accède à la valeur d'un ***key*** en l'utilisant comme index:
print(N[voiture]) affiche : '4 roues'

A21E8 Fonctions

Fonctions

Une fonction permet d'écrire un bloc de code que l'on va pouvoir appeler plusieurs fois à différent moment de notre programme sans avoir à le réécrire. Un fonction doit être définie avant de pouvoir être appelée. On définit une fonction à l'aide du mot clé ***def nom_de_ma_fonction(args) :*** on peut ensuite l'appeler par ***nom_de_ma_fonction()*** depuis notre programme principal ou depuis une autre fonction.
Une fonction peut prendre un ou plusieurs arguments en entrée et retourner un objet.
Exemple:

```
def saluer(nom):
    print(f'bonjour {nom}')
```

Utilisation de la fonction:

saluer('Philippe')
Saluer('marc')

Rendu du programme:

Bonjour Philippe
Bonjour marc

A21E9 Fonctions natives, fonction intégrée à python

abs	Retourne la valeur absolue d'un nombre
aiter	Renvoie un itérateur asynchrone
all	Retourne True si tous les éléments d'un itérable sont vrais
anext	Renvoie l'element suivant d'un iterable asynchrone
any	Retourne True si au moins un élément d'un itérable est vrai
ascii	Retourne une string qui représente l'objet
bin	Converti un nombre en binaire sous forme de string
bool	Converti en élément en valeur booléenne
breakpoint	Place dans le débugueur
bytearray	Retourne un tableau de taille donnée
bytes	Retourne un objet de type "byte"
callable	Test si l'objet est "appelable"
chr	Converti un nombre en sa valeur ascii sous forme de string
classmethod	Retourne la méthode de classe pour une fonction
compile	Exécute puis retourne du code Python
complex	Retourne un nombre complexe
delattr	Supprime un attribut de l'objet
dict	Crée et retourne un dictionnaire {}
dir	Retourne une liste des attributs d'un objet Python
divmod	Retourne un tuple avec le quotient et le reste d'une division
enumerate	Retourne un objet enumerate
eval	Exécute du code Python
exec	Exécute du code dynamiquement
filter	Crée un itérateur à partir d'éléments qui renvoient True
float	Retourne un nombre à virgule flottante
format	Retourne une représentation formatée d'une valeur
frozenset	Retourne un objet immutable fronzenset
getattr	Retourne la valeur de l'attribut nommé d'un objet
globals	Retourne un dictionnaire avec la table des symboles globaux
hasattr	Test si la l'objet a l'attribut ou pas
hash	Retourne un entier avec la valeur de hachage d'un objet
help	Appelle l'aide native de Python
hex	Converti un entier en nombre de base hexadécimale
id	Renvoie l'identification d'un objet
input	Lit et renvoie une ligne de chaînes de caractères
int	Converti un nombre ou une string en nombre entier
isinstance	Test si un objet est une instance de la classe
issubclass	Test si un objet est une instance d'une autre classe
iter	Retourne un itérateur
len	Retourne la longueur d'un objet
list	Crée une nouvelle liste ou transforme un autre objet en liste
locals	Retourne un dictionnaire avec la table des symboles locaux
map	Applique une fonction et retourne une liste
max	Retourne le plus grand élément
memoryview	Retourne une identification mémoire d'un objet de type byte
min	Retourne le plus petit élément
next	Retourne l'élément suivant d'un itérateur
object	Crée un objet Python
oct	Converti un entier en nombre de base 8
open	Retourne un objet de type fichier
ord	Retourne un entier d'un caractère avec sa valeur unicode
pow	Retourne la puissance d'un nombre
print	Permet d'afficher un objet sur la sortie standard
property	Retourne un objet "property"
range	Retourne une liste de nombres
repr	Retourne une représentation d'un objet
reversed	Retourne un itérateur dans le sens inverse
round	Retourne un nombre arrondi à une décimale donnée
set	Crée et retourne un set d'éléments uniques (sans doublon)
setattr	Défini la valeur d'un attribut de l'objet
slice	Coupe et retourne un objet
sorted	Trie et retourne un itérable
staticmethod	Transforme une méthode en une méthode statique
str	Retourne une chaîne de caractères
sum	Fait la somme des elements d'un iterable
super	Retourne un objet proxy de la classe
tuple	Crée et retourne un tuple
type	Retourne le type d'un objet Python
vars	Retourne l'attribut __dict__
zip	Prend des itérables et retourne une liste de tuples
__import__	Fonction appelé par l'instruction import

A21E10 Modules

Modules

Un module contient un ou plusieurs fichier python que l'on peut importer en début de script et qui contient un ensemble de définitions. Exemple le module ***time*** contient des fonctions permettant de dormir.
Exemple:

```
import time
time.sleep(3) # dort pendant 3 secondes
```

Un module spécifique à l'utilisation du raspi pico existe, c'est lui qui va nous permettre d'interagir avec les périphériques du microcontrôleur et donc avec le monde réel : ***machine***
Exemple:

from machine import Pin → depuis le module machine on déclare que l'on va utiliser l'objet Pin

P0 = Pin(0, Pin.OUT) → On déclare une variable P0 dans laquelle on range un abjet Pin particulier : la broche 0 du pico et on définit que ce sera une sortie

P0.value(1) → On peut maintenant utiliser les fonctions propre à l'objet Pin, ici la fonction (méthode) value auquel on passe un argument (paramètre) '1' signifiant que l'on souhaite que la broche prenne un état électrique haut (vcc)

- **Application : Le code Morse**

  Présentation

  La cadence

  Câblage des 2 LEDs

  Le programme

Le code Morse

Le **code Morse international**[1], ou l'**alphabet Morse international**, est un code universel permettant de transmettre un texte à l'aide de séries d'impulsions courtes et longues, qu'elles soient produites par des signes, une lumière, un son ou un geste.

Le matériel (version simple)





Une clé, un buzzer ou une LED
(avec un microcontrôleur comme un Raspberry Pi Pico)
et c'est parti !

Exemple de message

• • • — — — • • •

S O S



Message transmis en Scott (Morse lumineux), par un timonier, au moyen d'un projecteur (US Navy, mars 2005).

Le code Morse

Lettres de l'alphabet

Lettre	Code	Lettre	Code
A	• –	N	– •
B	– • • •	O	– – –
C	– • – •	P	• – – •
D	– • •	Q	– – • –
E	•	R	• – •
F	• • – •	S	• • •
G	– – •	T	–
H	• • • •	U	• • –
I	• •	V	• • • –
J	• – – –	W	• – –
K	– • –	X	– • • –
L	• – • •	Y	– • – –
M	– –	Z	– – • •

Chiffres

Chiffre	Code
1	• – – – –
2	• • – – –
3	• • • – –
4	• • • • –
5	• • • • •
6	– • • • •
7	– – • • •
8	– – – • •
9	– – – – •
0	– – – – –

Quelques signes de ponctuation

Signe	Code
.	• – • – • –
,	– – • • – –
?	• • – – • •
/	– • • – •
(	– • – – •
)	– • – – • –
:	– – – • • •
;	– • – • – •
=	– • • • –
'	• – – – – •
@	• – – • – •

La cadence

Les LED

LED Morse (GPIO 7)
s'allume pour les points et les traits

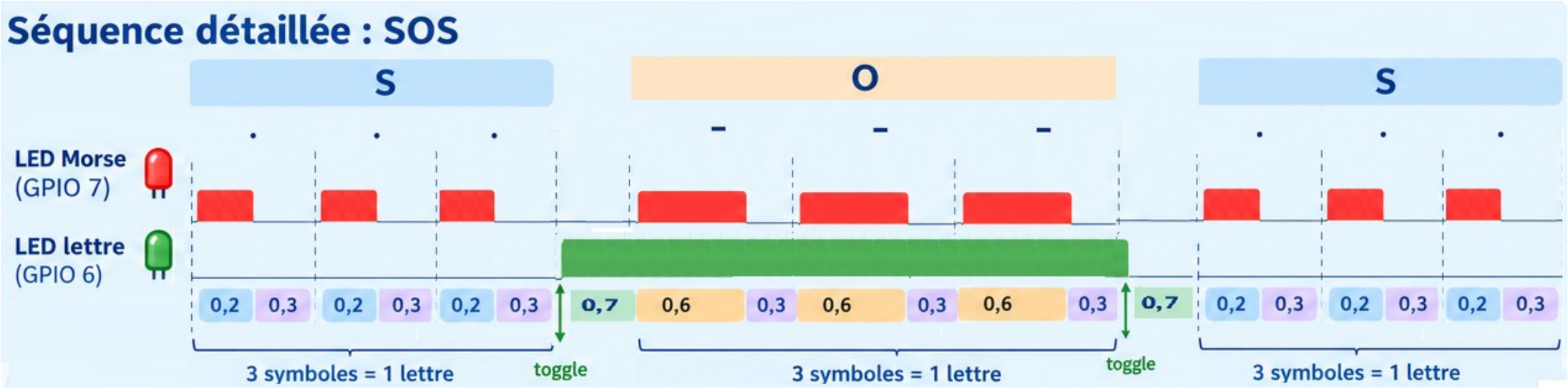
LED lettre (GPIO 6)
change d'état (toggle) à chaque changement de lettre

Les durées

POINT_TIME	= 0,2 s	durée d'un point (.)
TIRET_TIME	= 0,6 s	durée d'un trait (-)
PAUSE_SYMBOLE	= 0,3 s	après chaque symbole (même le dernier)
PAUSE_LETTRE	= 0,7 s	après les 3 symboles d'une lettre

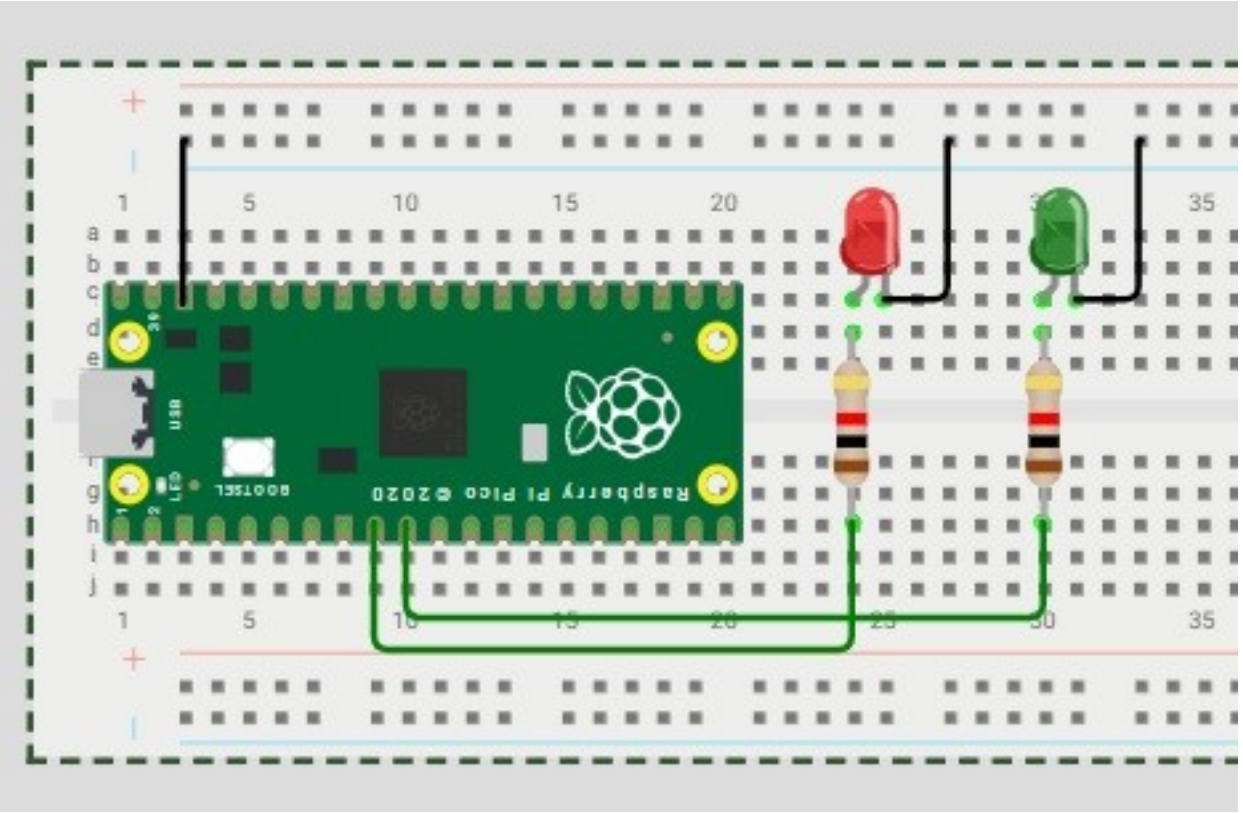
- **Point (.)** : 0,2 s allumé + 0,3 s éteint
- **Trait (-)** : 0,6 s allumé + 0,3 s éteint

Séquence détaillée : SOS



Câblage des 2 LEDs

* Faire vérifier le montage avant connexion du microcontrôleur au PC
* Ne pas mettre les doigts sur le circuit imprimé du microcontrôleur
* Tension max sur chaque Pin = 3,3V en Entrée comme en Sortie



Composant	GPIO
Led 1 (rouge)	6
Led 2 (verte)	7

Code couleur des câbles :
- Vcc : Rouge / Orange
- Gnd : Noir / Marron
- Données : Vert / Bleu / Jaune / Violet

Le programme

```python
from machine import Pin
import time

time.sleep(0.1)  # Attendre que l'USB soit prêt

print("Hello, Pi Pico!")

# LEDS
led_lettre = Pin(6, Pin.OUT)
led_morse = Pin(7, Pin.OUT)

# TIMING MORSE
POINT_TIME = 0.2
TIRET_TIME = 0.6
PAUSE_SYMBOLE = 0.3
PAUSE_LETTRE = 0.7

# CODE MORSE
S = (".", ".", ".")
O = ("-", "-", "-")

# Message : SOS
message = (S, O, S)

# INITIALISATION
led_lettre.value(0)
led_morse.value(0)
```

```python
print("debut du message")

# ENVOI DU MESSAGE

for lettre in message:

    for symbole in lettre:

        # Allumer la LED Morse
        led_morse.value(1)

        if symbole == "-":
            time.sleep(TIRET_TIME)
        else:
            time.sleep(POINT_TIME)

        # Éteindre la LED Morse
        led_morse.value(0)

        # Pause entre deux symboles
        time.sleep(PAUSE_SYMBOLE)

    # Indiquer le changement de lettre
    led_lettre.toggle()

    # Pause supplémentaire entre deux lettres
    time.sleep(PAUSE_LETTRE)

print("fin du message")
```